Assignment 6

Aarav Dudeja
102303179

Q1)

```
%%writefile device_query.cu
#include <iostream>
#include <cuda_runtime.h>

int main() {
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);

    for (int i = 0; i < deviceCount; i++) {
        cudaDeviceProp prop;
        cudaGetDeviceProperties(&prop, i);
        std::cout << "Device Name: " << prop.name << std::endl;
        std::cout << "Compute Capability: " << prop.major << "." << prop.minor << std::endl;
        std::cout << "Max Grid Dimensions: (" << prop.maxGridSize[0] << ", " << prop.maxGridSize[1] << ", " << prop
        std::cout << "Max Block Dimensions: (" << prop.maxThreadsDim[0] << ", " << prop.maxThreadsDim[1] << ", " <<
        std::cout << "Max Threads Per Block: " << prop.maxThreadsPerBlock << std::endl;
        std::cout << "Total Global Memory: " << prop.totalGlobalMem / (1024 * 1024) << " MB" << std::endl;
        std::cout << "Shared Memory per Block: " << prop.sharedMemPerBlock / 1024 << " KB" << std::endl;
        std::cout << "Constant Memory: " << prop.totalConstMem / 1024 << " KB" << std::endl;
        std::cout << "Warp Size: " << prop.warpSize << std::endl;
        std::cout << "Double Precision Support: " << (prop.major >= 1 ? "Yes" : "No") << std::endl;
    }
    return 0;
}
```

```
Writing device_query.cu
```

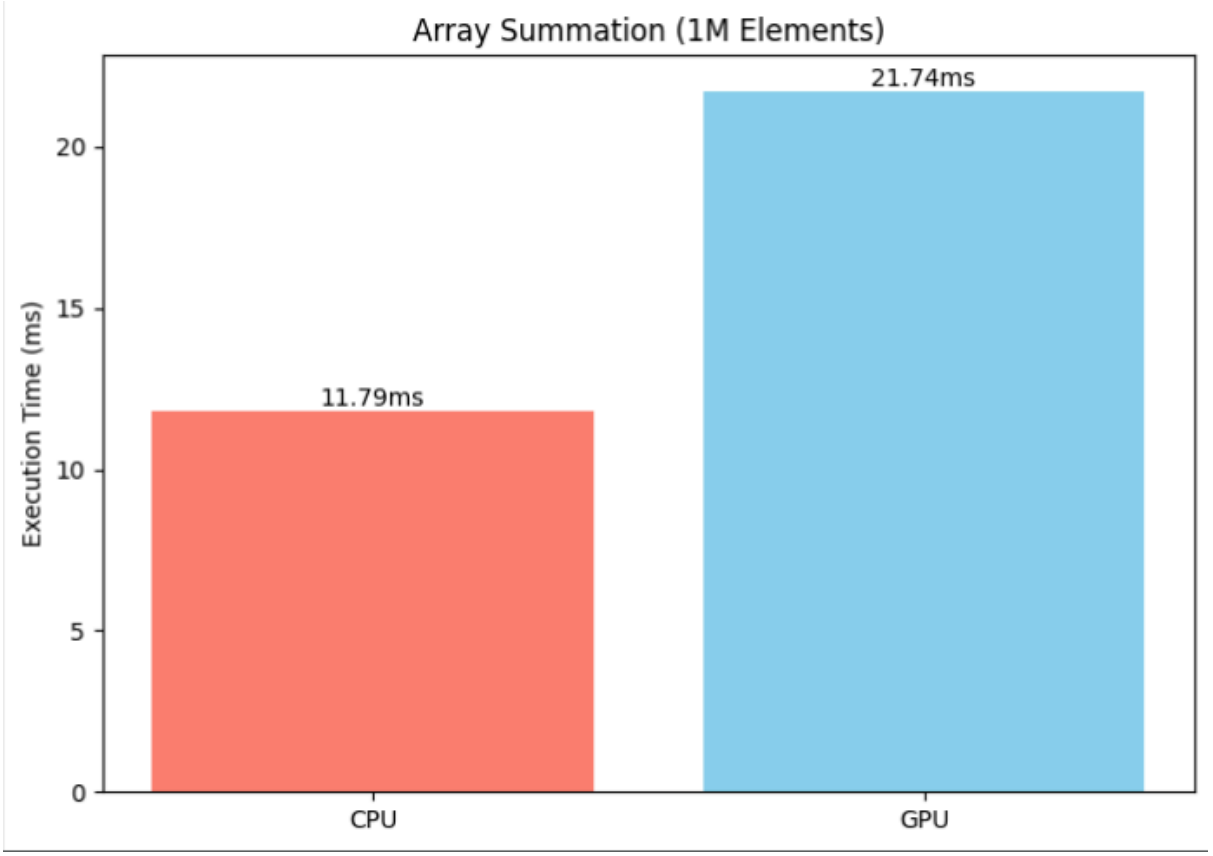
```
!nvcc device_query.cu -o device_query && ./device_query
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a
Device Name: Tesla T4
Compute Capability: 7.5
Max Grid Dimensions: (2147483647, 65535, 65535)
Max Block Dimensions: (1024, 1024, 64)
Max Threads Per Block: 1024
Total Global Memory: 14912 MB
Shared Memory per Block: 48 KB
Constant Memory: 64 KB
Warp Size: 32
Double Precision Support: Yes
```

1. **Architecture & Compute Capability**: The GPU is a Tesla T4 with Compute Capability 7.5.
2. **Max Block Dimensions**: (1024, 1024, 64).
3. **Max Threads Calculatio**n: With a max grid dimension of 65,535 and a max block dimension of 512, the maximum number of threads in a 1D configuration is 65535*512 = 33,553,920
4. **Limiting Threads**: Programmers may avoid the maximum to reduce overhead on small datasets or to ensure each thread has enough registers and shared memory to maintain high occupancy.
5. **Limits**: Hardware constraints like registers per Streaming Multiprocessor (SM), shared memory per SM, and maximum threads per block limit the total active threads.

6. **Shared Memory**: It is a fast, on-chip memory shared by threads in a block. Your GPU has 48 KB per block.
7. **Global Memory**: The main GPU DRAM accessible by all threads. Your GPU has 14,912 MB.
8. **Constant Memory**: Read-only cached memory for fixed values. Your GPU has 64 KB.
9. **Warp Size**: This is the smallest unit of execution; threads in a warp execute in lock-step. Your GPU's warp size is 32.
10. **Double Precision**: Yes, it is supported

Q2)



GPU(CUDA) vs CPU timings for Q2

```
%%writefile array_sum.cu
#include <iostream>
#include <vector>

__global__ void sumKernel(float* d_in, float* d_out, int n) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < n) ? d_in[i] : 0.0f;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) d_out[blockIdx.x] = sdata[0];
}

int main() {
    int n = 1024;
    size_t size = n * sizeof(float);
    std::vector<float> h_in(n, 1.0f);
    float h_out = 0;

    float *d_in, *d_out;
    cudaMalloc(&d_in, size);
    cudaMalloc(&d_out, sizeof(float));

    cudaMemcpy(d_in, h_in.data(), size, cudaMemcpyHostToDevice);

    sumKernel<<<1, n, n * sizeof(float)>>>(d_in, d_out, n);

    cudaMemcpy(&h_out, d_out, sizeof(float), cudaMemcpyDeviceToHost);

    std::cout << "Sum: " << h_out << std::endl;

    cudaFree(d_in);
    cudaFree(d_out);
    return 0;
}
```

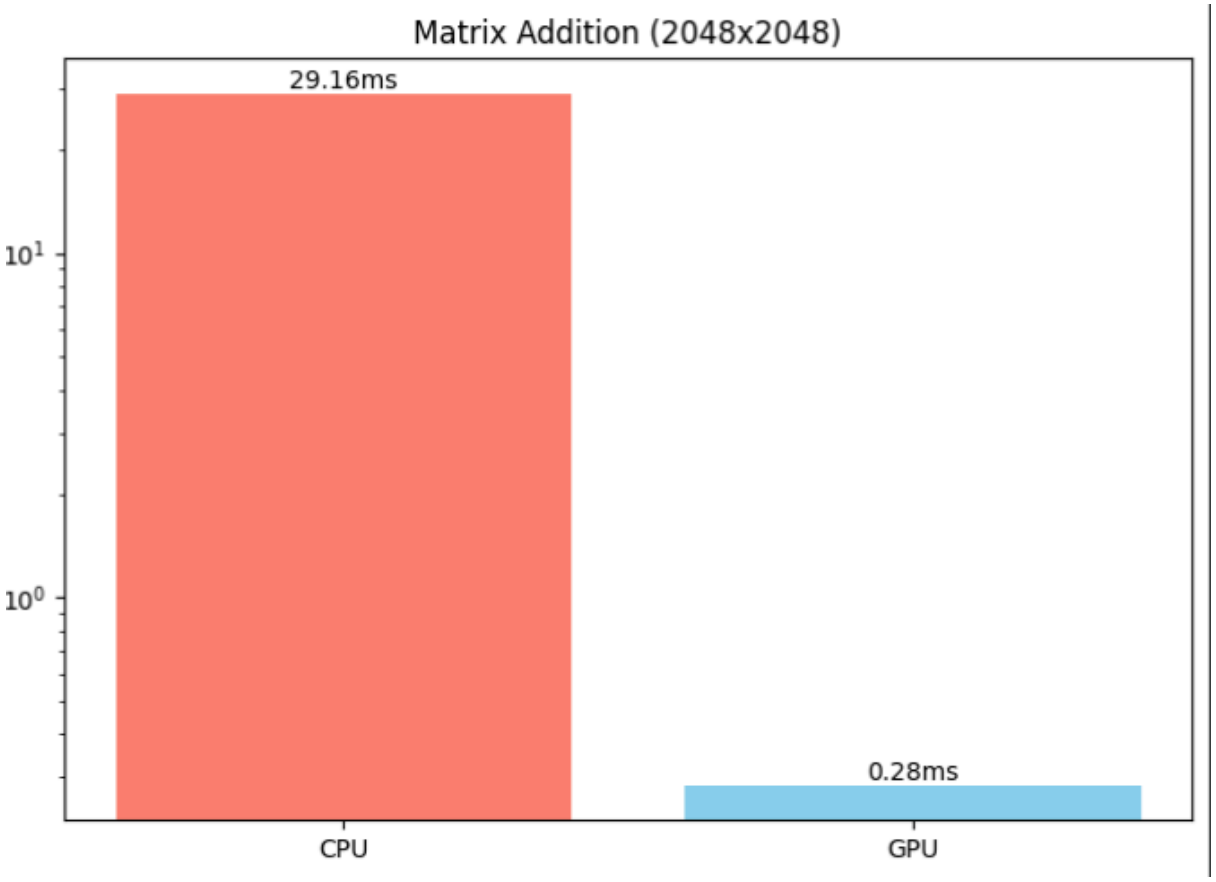
```
!nvcc array_sum.cu -o array_sum && ./array_sum

nvcc warning : Support for offline compilation for architectures
Sum: 1024
```

Q3)

```
%%writefile matrix_add.cu
#include <iostream>

__global__ void matrixAdd(int* A, int* B, int* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        int idx = row * N + col;
        C[idx] = A[idx] + B[idx];
    }
}

int main() {
    int N = 1024;
    size_t size = N * N * sizeof(int);
    int *h_A, *h_B, *h_C;
    h_A = (int*)malloc(size);
    h_B = (int*)malloc(size);
    h_C = (int*)malloc(size);

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    dim3 threads(16, 16);
    dim3 grid((N + 15) / 16, (N + 15) / 16);

    matrixAdd<<<grid, threads>>>(d_A, d_B, d_C, N);
    cudaDeviceSynchronize();

    std::cout << "Matrix addition completed successfully." << std::endl;

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);
    return 0;
}
```

```
!nvcc matrix_add.cu -o matrix_add && ./matrix_add
```

```
nvcc warning : Support for offline compilation for archi
Matrix addition completed successfully.
```



GPU(CUDA) vs CPU timings for Q3

- **Operations**: 0 floating point operations (this is integer addition). If they were floats, it would be N additions
- **Global Memory Reads**: $2 \times N^2$ (reading A and B). C.
- **Global Memory Writes**: $1 \times N^2$ (writing C).